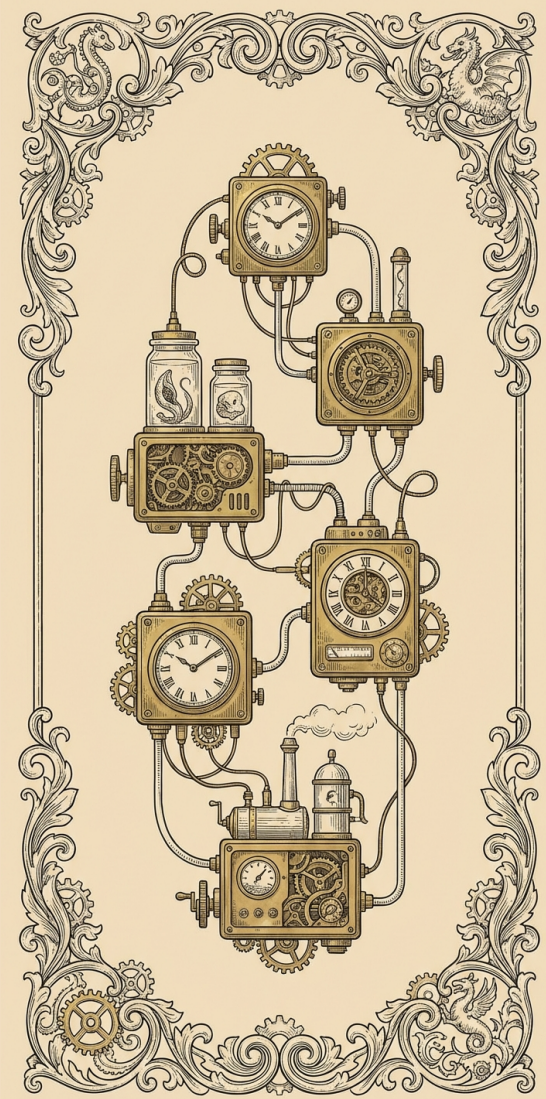
CS 3960 (3)

Vibe Coding

Spring 2026, *content* Pavel Panchekha and John Regehr, *illustrations* Gemini 3 Nano Banana Pro

- 2.5 weeks of classes left after today
- You have assignments due April 3, 10, 17
- Let's talk about grading
- Questions about last assignment?
- Reading assignment: On The Criteria To Be Used In Decomposing Systems Into Modules

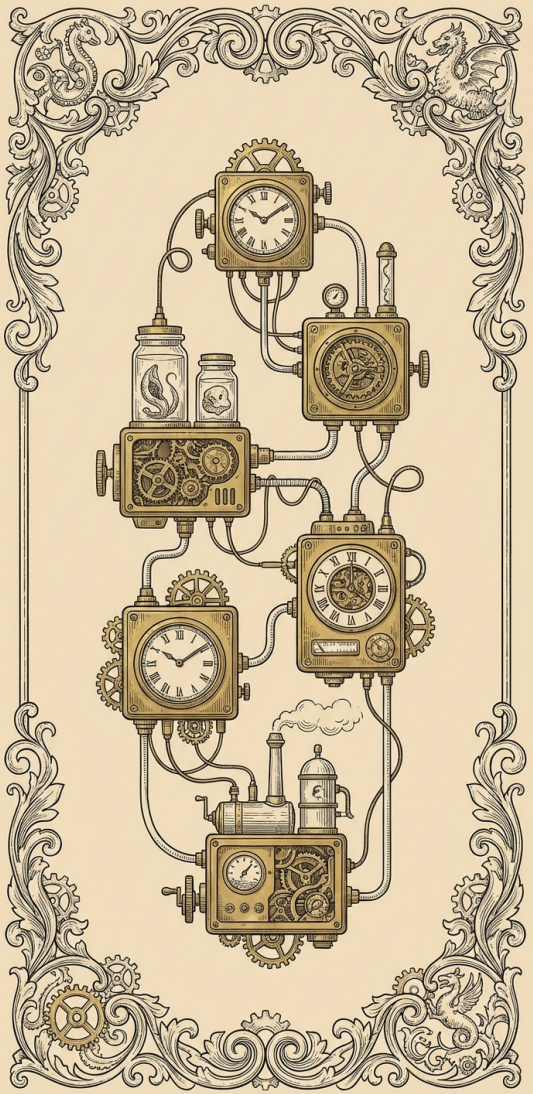


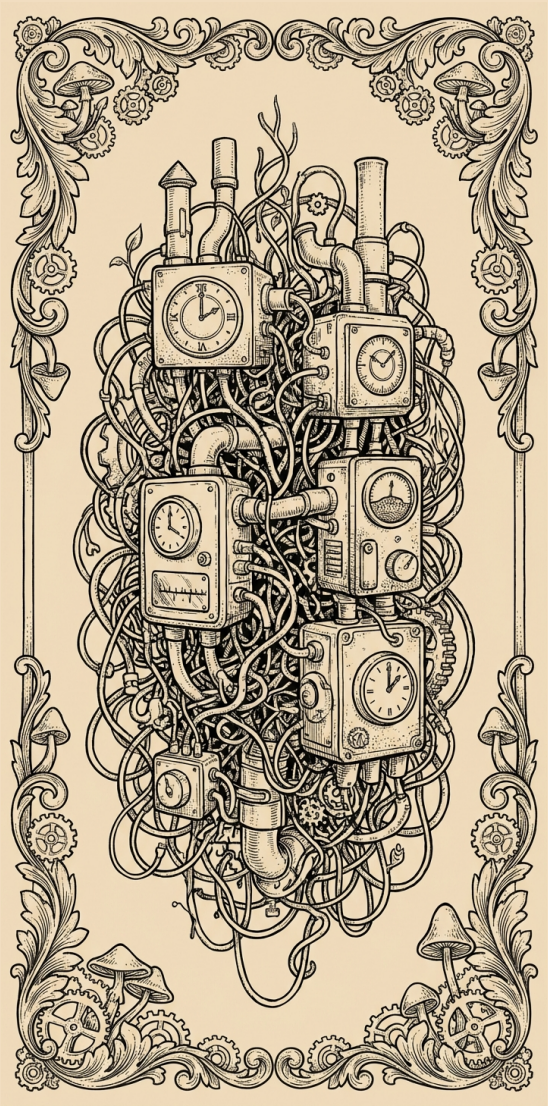
Today:

Software Modularity and Interfaces

Part of a modular software system can be independently…

- Understood
- Changed
- Replaced



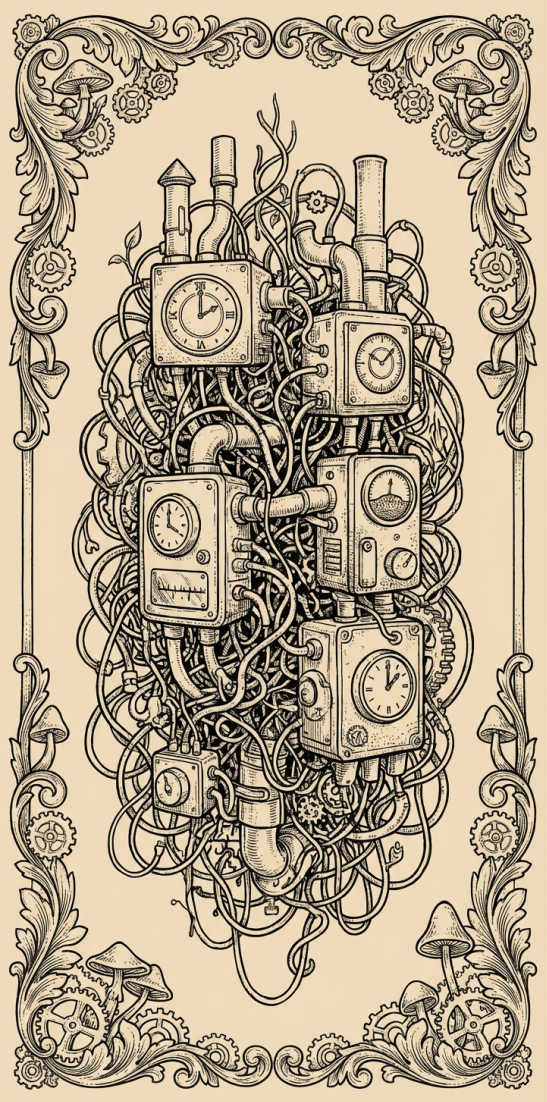


When software is modularized poorly, we cannot independently

- Understand part of it
- Change part of it
- Replace part of it

As systems scale up, modularity becomes one of the most important things

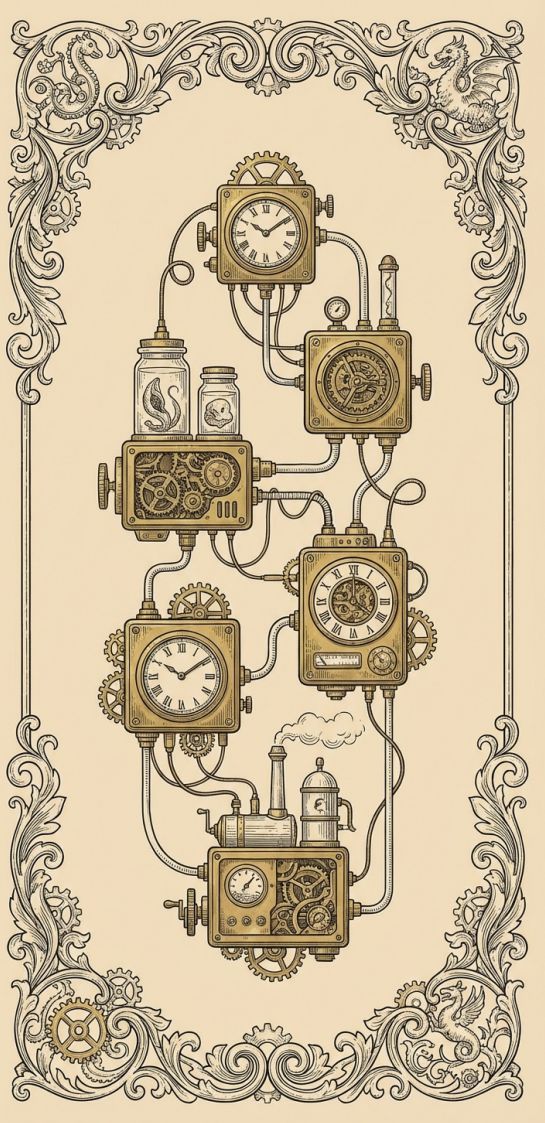
You're very unlikely to ever understand Google Chrome, but with some work you could understand one of its modules

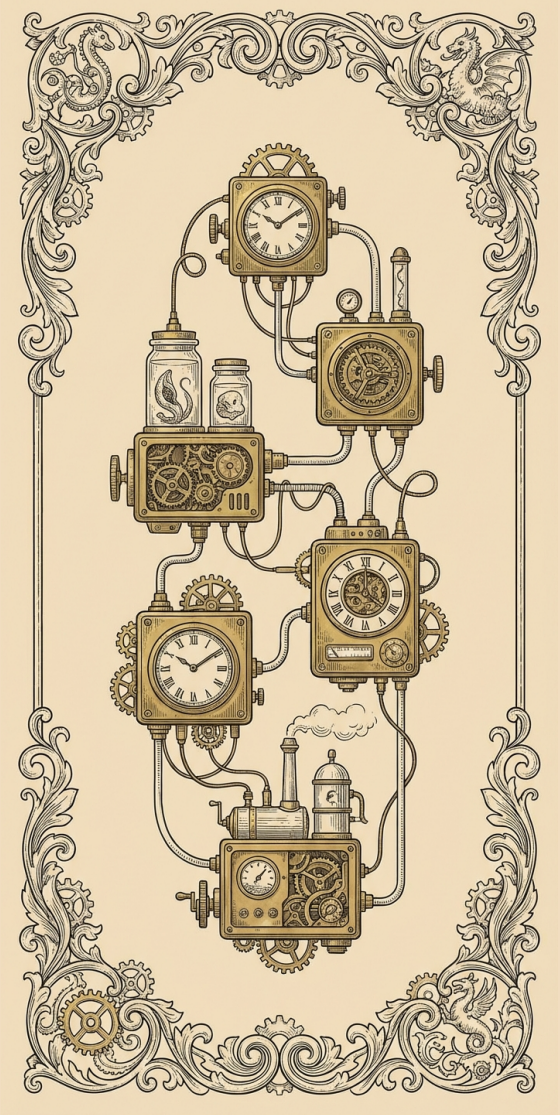


Modularity in the LLM age...

Coding agents can (and do) respect modularity — but you may have to keep reminding them

Coding agents are not great at decomposing a system into modules





Modularity is the key to software engineering at scale

Modularity is what allows us to break a complex software problem into pieces

The question isn't "should we write modular software"

The question is how to best achieve this

What are some ways to modularize a text editor?

The basic thing that makes software modular is the **interface**

An interface is how you use a piece of software

Has little to do with how the software is actually implemented

Interface design is hard

Modern programming languages often have explicit support for interfaces

This only makes interfaces look nicer, it doesn't make designing interfaces easy

A good interface...

- Hides irrelevant details
- Exposes details that are necessary
- Is difficult to use incorrectly

Good interfaces are hard to design

There are a lot of bad interfaces out there

- Encryption libraries
- The X windows library
- SOAP — simple object access protocol

What do interfaces look like in code?

It might just be function calls!

“Procedural decomposition” is the most basic kind of modularity

Modern programming languages usually have explicit support for interfaces

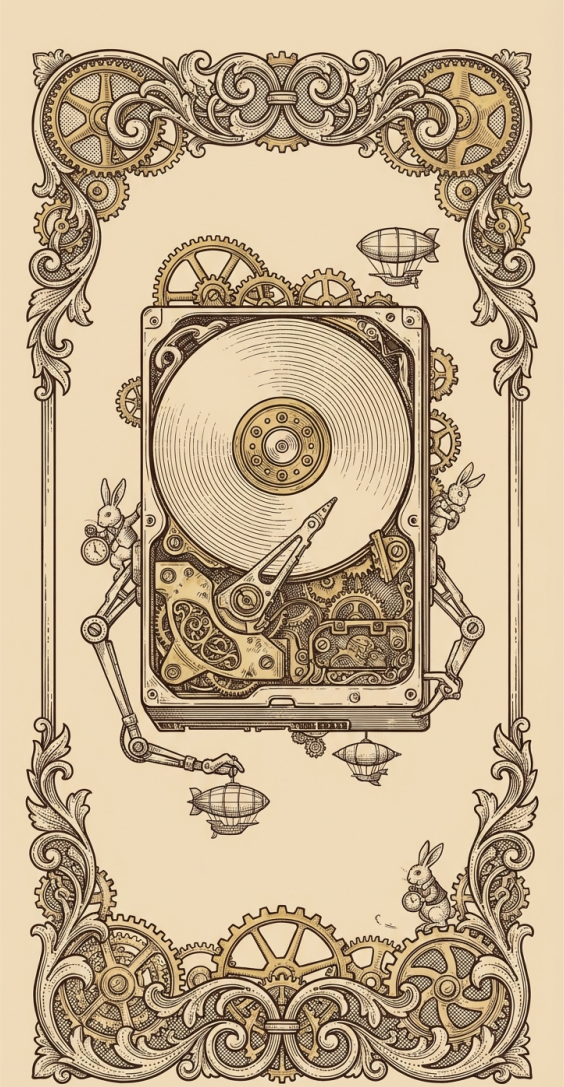
This only makes interfaces look nicer — doesn't make designing interfaces easy!

An interface might...

- Call into a different programming language
- Call into a different process
- Call out across the network
- ....

The form that an interface takes is unimportant

What's important is that we have interfaces between software modules



Let's talk about designing an interface to a persistent storage device

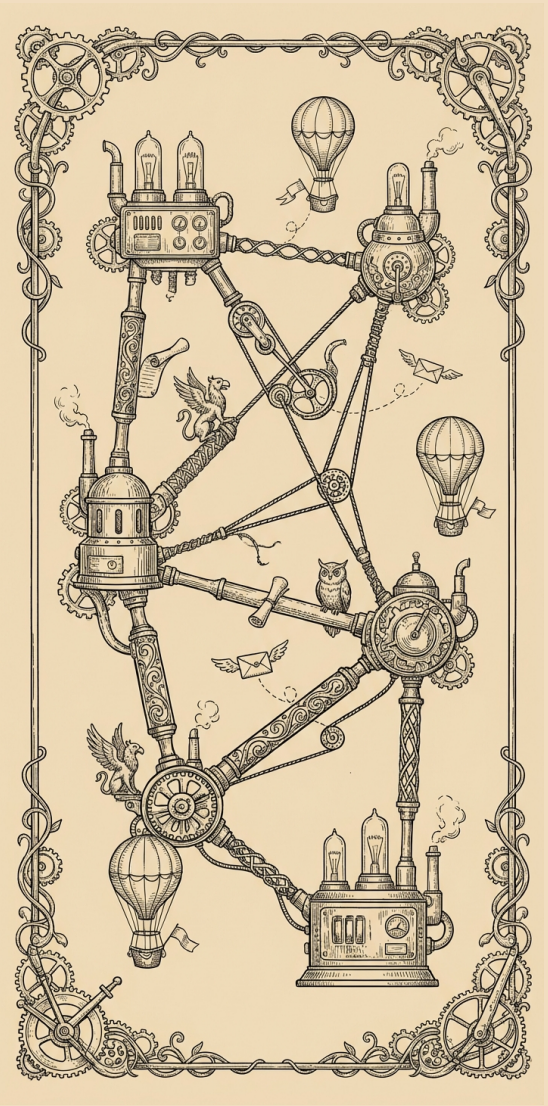
What should we expose?

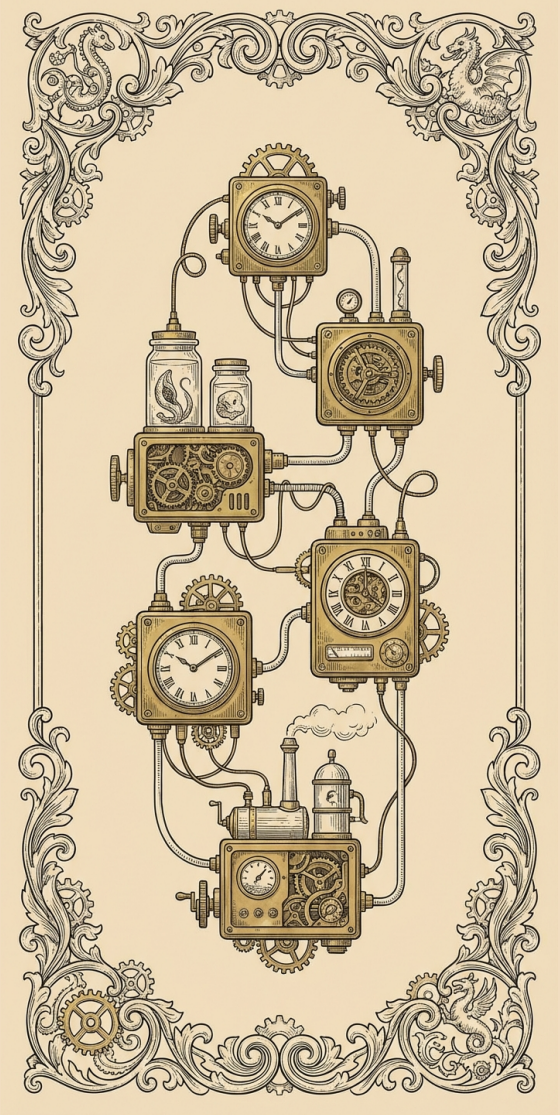
What should we hide?

Let's talk about designing an interface to a computer network

What should we expose?

What should we hide?





Important abstractions tend to be leaky

That is, they only incompletely hide what they're trying to hide

... this is just a fact of life

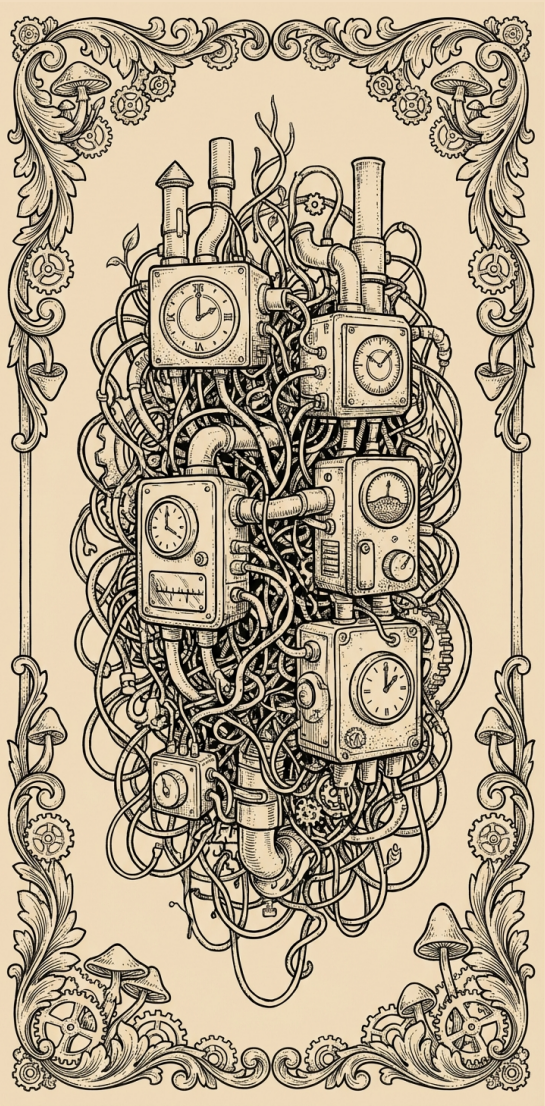
Modularity can be enforced

At security boundaries we must do this

But we can also do it at other module boundaries, to protect against accidental misuse

Example:

The Linux / Windows / OS X kernels are protected from user programs even if there's only one user

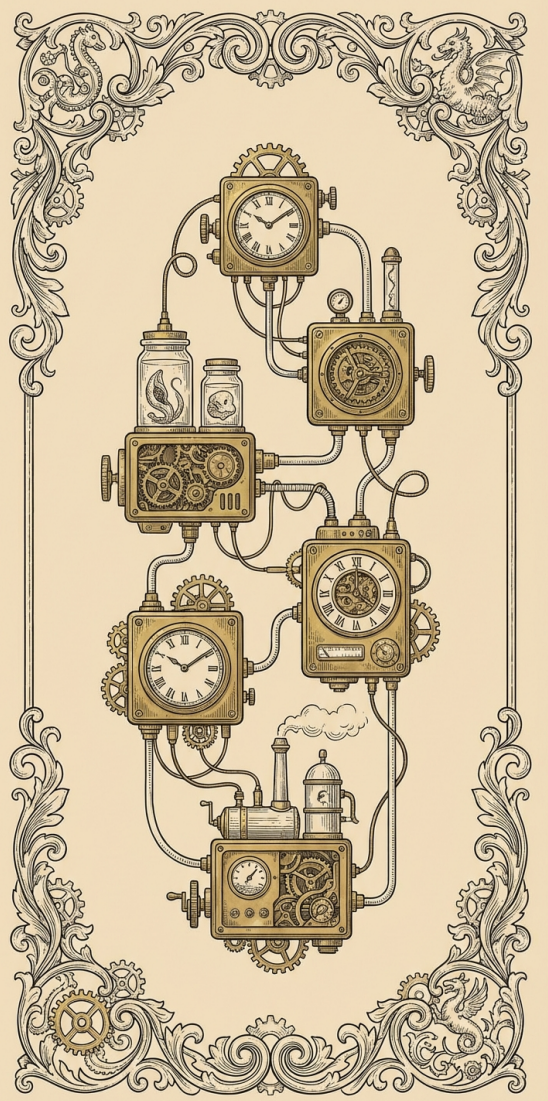


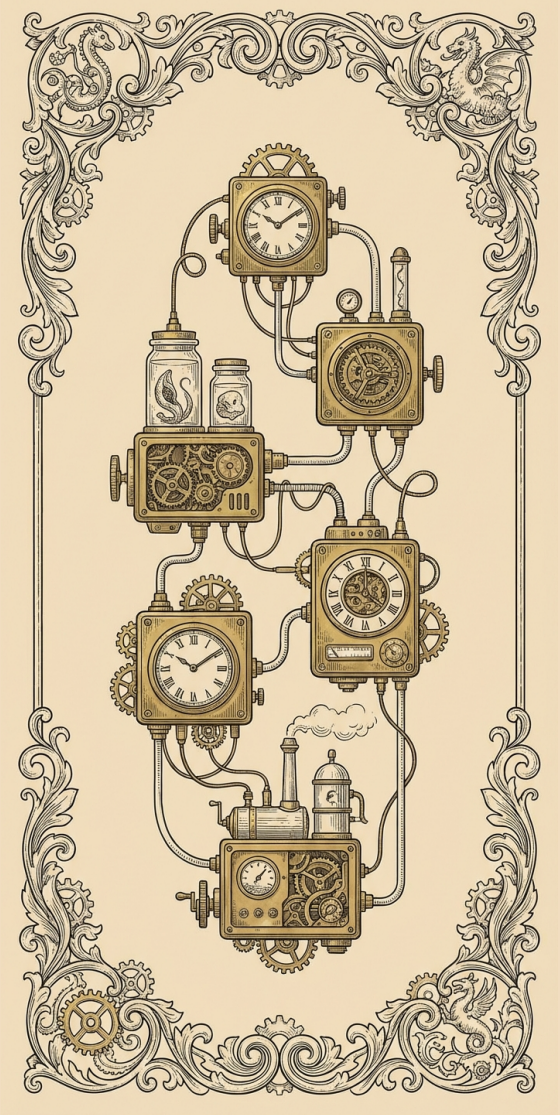
Some systems appear to be modular, but actually aren't

The Linux kernel is decomposed into modules — but in practice it has pointers going everywhere, inside the kernel you can read or write any data

We sometimes care about things that are hard to modularize

- Errors and failures
- Security concerns
- Power management in device drivers





Summary:

With or without LLMs, we can't engineer large software without modularity

As software gets bigger, modularity is more important than ever

You'll have to be in charge of modularity for your LLM-assisted projects